

Изучение OAuth2: подключение провайдеров идентификации в приложение

Куляка Павел Дмитриевич

Группа: 25.Б41-мм

Кафедра: Системного программирования

Научный руководитель: Смирнов Кирилл Константинович

Номер семестра практики: 2

1. Цель работы

Целью данной работы является изучение протоколов OAuth 2.0 и OpenID Connect, а также практическая реализация двух взаимодействующих приложений — сервера авторизации (OIDC-провайдера) и клиента с возможностью входа через несколько внешних провайдеров, реализующих процесс получения токена по коду авторизации (Authorization Code Flow).

Протоколы OAuth 2.0 и OIDC используются для авторизации сторонних сервисов для доступа к данным и для аутентификации пользователей. Протокол OAuth 2.0 позволяет стороннему приложению получить ограниченный доступ к ресурсам пользователя без передачи ему пароля. OpenID Connect (OIDC) — надстройка над OAuth 2.0, добавляющая слой аутентификации: сервер авторизации возвращает клиенту ID Token в формате JWT, содержащий информацию о пользователе. OIDC также определяет механизмы Discovery (автоматическое получение конфигурации сервера) и JWKS (публикация ключей для проверки подписи токенов).

2. Реализация

2.1. Архитектура

Проект состоит из двух независимых Flask-приложений: клиента и провайдера. Клиент выступает в роли OAuth-клиента и предоставляет пользователю интерфейс для входа через одного из поддерживаемых провайдеров. Провайдер реализует полноценный сервер авторизации OAuth 2.0 с расширением OpenID Connect.

Реализация решения осуществлялась с использованием языка Python, веб-фреймворка Flask, библиотек PyJWT (работа с JWT-токенами), sturptography (криптографические операции), Werkzeug (хэширование паролей) и Requests (HTTP-запросы к провайдерам). Интерфейс свёрстан с использованием TailwindCSS.

2.2. Клиент

Клиент поддерживает авторизацию через четырёх провайдеров:

- Google — протокол OpenID Connect (получение ID Token, проверка подписи через JWKS).
- GitHub — протокол OAuth 2.0 (получение Access Token, запрос email через API).
- Yandex — протокол OAuth 2.0 (аналогично GitHub).
- My Service — собственный OpenID Connect-провайдер.

2.2.1. Особенности подключения внешних провайдеров

При подключении сторонних сервисов был учтён ряд особенностей. В экосистеме Яндекса существует два сервиса: Yandex ID (не поддерживает OIDC) и Yandex Identity Hub (поддерживает OIDC, PKCE, JWKS и Discovery). В работе был использован Yandex ID через OAuth 2.0 без OIDC. Также изначально вместо GitHub планировалось подключить авторизацию через Mail.ru, но, поскольку для регистрации приложения требовалось юридическое лицо, этот вариант был исключён.

2.3. Провайдер

Провайдер реализует полный процесс авторизации. Поддерживаются следующие endpoint'ы:

- /oauth/.well-known/openid-configuration — OpenID Connect Discovery.
- /oauth/authorize — endpoint авторизации; создаёт код авторизации после успешной авторизации пользователя.
- /oauth/token — token endpoint; обменивает код авторизации на Access Token и ID Token (JWT, подписанный RS256).
- /oauth/userinfo — возвращает информацию о пользователе по действительному токenu.
- /oauth/jwks.json — публикует JWK Set для проверки подписи ID Token.

ID Token формируется как JWT. Подпись выполняется алгоритмом RS256 с использованием RSA-ключа длиной 2048 бит.

Для хранения данных (пользователи, зарегистрированные приложения, коды авторизации, токены) используется SQLite. Пароли пользователей хранятся в хэшированном виде (Werkzeug).

3. Тестирование

Для проверки корректности реализации написаны модульные тесты с использованием pytest. Покрыты следующие сценарии:

- Авторизация через всех провайдеров (проверка формирования URL авторизации, корректность параметров state/nonce).
- Обмен кода авторизации на токены, в том числе с PKCE (plain и S256).
- Проверка подписи ID Token через JWKS.
- Валидация обязательных параметров и обработка ошибочных запросов (неверный client_id, просроченный код, неверный redirect_uri).
- Регистрация пользователей и OAuth-приложений на стороне провайдера.
- Управление приложениями (удаление, регенерация client_secret).
- Вспомогательные функции: распознавание email, извлечение email из структур данных, построение URL и заголовков.

4. Качество кода и CI

Также настроен CI через GitHub Actions, включающий три workflow: запуск тестов (pytest), линтинг (ruff) и проверка типов (myru).

5. Апробация

Была проведена ручная интеграционная проверка: клиент подключался к OIDC-провайдеру научного руководителя. Проверялся полный цикл — редирект с клиента на провайдера, ввод логина и пароля на странице авторизации провайдера, получение авторизационного кода, редирект обратно на клиент, обмен кода на токены и валидация ID-токена (проверка подписи, issuer, audience и nonce). Среди недочетов были выявлены сложность добавления новых провайдеров, проблемы с подписью токенов, которые были решены добавлением единого файла конфигурации провайдеров.

6. Выездной день

В рамках учебной практики состоялся выездной день в офис компании YADRO. Основной целью мероприятия было знакомство студентов с деятельностью компании, её продуктами и возможностями профессионального развития для молодых специалистов.

Представители компании рассказали о ключевых направлениях работы YADRO, о разрабатываемых продуктах и о задачах, которые решают инженерные команды. Особенно интересной для меня оказалась тема систем хранения данных: было полезно узнать, как обеспечиваются надёжность, производительность и отказоустойчивость таких систем.

Лекции были построены не только как обзор деятельности компании, но и как небольшое введение в реальные инженерные задачи. Спикеры объясняли основные принципы работы технологий и делились практическими примерами. Отдельное внимание уделялось вопросам измерения производительности: рассказывали о выборе метрик, поиске причин снижения эффективности и способах устранения «узких мест» в системе.

Также нам подробно рассказали о возможностях для студентов. Особое внимание было уделено стажировке «Импульс», на которую можно попасть уже со второго курса, а также другие направления для начинающих специалистов. Кроме того, сотрудники упомянули практические курсы и образовательные программы, которые компания проводит для студентов в течение года.

В конце мероприятия прошла встреча со стажёрами компании, среди которых были выпускники и студенты математико-механического факультета СПбГУ. Они поделились своим опытом работы и обучения, рассказали о проектах, в которых участвуют, и ответили на вопросы о стажировках и работе в компании. Было интересно услышать, как знания, полученные в университете, применяются на практике.

Выездной день оставил положительное впечатление. Мероприятие помогло лучше представить, как устроена работа в крупной IT-компании, и дало более понятное представление о возможных направлениях дальнейшего профессионального развития.

7. Заключение

В ходе выполнения работы получены следующие результаты:

- Реализован сервер авторизации (провайдер) OAuth 2.0 / OpenID Connect;
- Реализовано клиентское веб-приложение с поддержкой входа через Google (OIDC), GitHub (OAuth 2.0), Яндекс (OAuth 2.0) и локальный OIDC-провайдер;
- Разработана система управления пользователями и приложениями (регистрация, авторизация, удаление приложений, сброс секретов);
- Написаны модульные тесты, настроена непрерывная интеграция через GitHub Actions;
- Проведена интеграционная проверка с OIDC-провайдером научного руководителя, подтвердившая совместимость реализации;
- Реализована поддержка множества внешних провайдеров через единый интерфейс конфигурации, что позволяет легко добавлять новых провайдеров путём описания их параметров в конфигурационном словаре.

Репозиторий: <https://github.com/pavelkuliaka/educational-practice>